



Course: **EE321 Intro to AI**
Name: **Hamza Ahmad**

Complex Engineering Problem
Reg.: **2023217**

IMPORTANT: This is an **INDIVIDUAL** project

You must run the code yourself.

The exam will ask specific questions about **YOUR** model's behavior.
If you copy from the internet, you will likely fail the viva/exam.

Your Must

Build an appropriate AI model that can tell whether a picture contains a **hot dog** or **not a hot dog**.

Learning Objectives

By the end of this project, you will be able to:

- Load and explore image data
- Build an ANN and a CNN in TensorFlow/Keras
- Identify and fix overfitting using Dropout and Data Augmentation
- Plot and interpret training curves
- Test your model on real-world images

Dataset

You will use the **Hot Dog / Not Hot Dog** dataset from Kaggle:

<https://www.kaggle.com/dansbecker/hot-dog-not-hot-dog>

What You Need

- A Google account (for Google Colab)
- The dataset downloaded to your computer (upload to Colab)
- **Submit your overleaf generated PDF by May 07, 2026 by 12:00 PM**

Grading Rubric

Task	Points
Step 2: Data Exploration (answers)	5
Step 3: Visualization (screenshot)	5
Step 5: ANN Model (accuracy + analysis)	10
Step 6: CNN Model (accuracy + comparison)	15
Step 7: Overfitting Fix (before/after plots)	20
Step 8: Real Image Test (screenshot)	10
Step 9: Answers to all questions	20
Step 10: YOLO Extension (extra credit)	5
Total	85 + 5 EC

Step 1: Setup Google Colab

How to Run This Project in Google Colab

1. Go to <https://colab.research.google.com>
2. Click **File** → **New Notebook**
3. Rename it: `HotDog_Classifier_YourName.ipynb`
4. For each code block below, copy the code into a cell and press **Shift + Enter** to run
5. To upload the dataset: Click the folder icon, upload `hot-dog-not-hot-dog.zip`, then run `!unzip hot-dog-not-hot-dog.zip`

Step 2: Import Libraries

Run this code in Colab:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import os
import random

print("Libraries loaded successfully!")
print("TensorFlow version:", tf.__version__)
```

1. What TensorFlow version are you using?

Answer: TensorFlow version: **2.20.0**

Step 3: Load and Explore the Data

Run this code to see how many images you have:

```
# Define paths (CHANGE THIS to your actual path)
train_path = "/content/hot-dog-not-hot-dog/train"
test_path = "/content/hot-dog-not-hot-dog/test"

# Count images
hot_dog_count = len(os.listdir(os.path.join(train_path, "hot_dog")))
not_hot_dog_count = len(os.listdir(os.path.join(train_path, "
    not_hot_dog")))
print(f"Hot dog images: {hot_dog_count}")
print(f"Not hot dog images: {not_hot_dog_count}")

# YOUR CODE HERE: Print the number of test images
hot_dog_test = len(os.listdir(os.path.join(test_path, "hot_dog")))
not_hot_dog_test = len(os.listdir(os.path.join(test_path, "not_hot_dog
    ")))
print(f"Test Hot dog images: {hot_dog_test}")
print(f"Test Not hot dog images: {not_hot_dog_test}")
```

1. How many training images are in each class?

Answer: Hot dog images: **249**

Not hot dog images: **249**

2. Is the dataset balanced? Why does that matter?

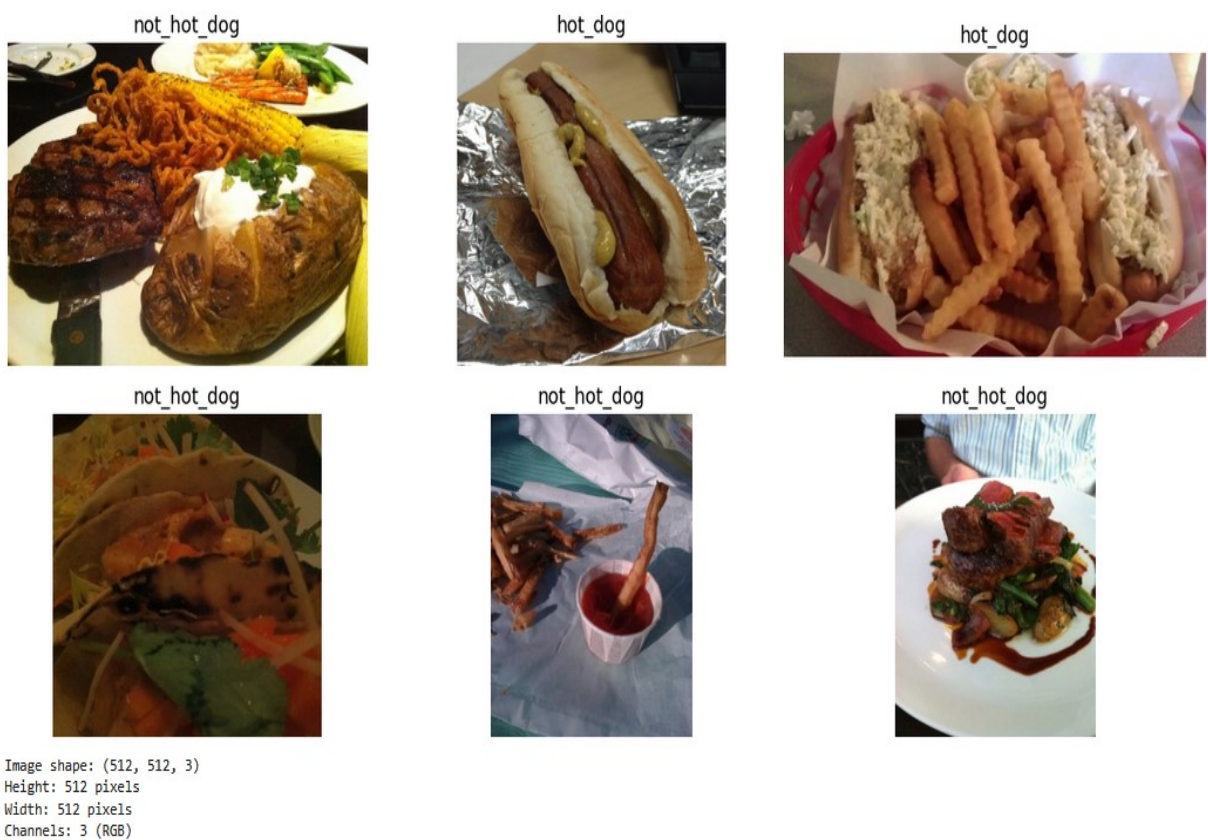
Answer: Yes, the dataset is balanced because both classes contain equal number of images. This prevents bias toward one class.

Step 4: Visualize the Data

```
# Display 6 random images
plt.figure(figsize=(12, 6))
for i in range(6):
    class_name = random.choice(["hot_dog", "not_hot_dog"])
    class_path = os.path.join(train_path, class_name)
    img_name = random.choice(os.listdir(class_path))
    img_path = os.path.join(class_path, img_name)
    img = plt.imread(img_path)
    plt.subplot(2, 3, i+1)
    plt.imshow(img)
    plt.title(class_name)
    plt.axis('off')
plt.tight_layout()
plt.show()

# YOUR CODE HERE: Print the shape of one image
print(img.shape)
```

1. Paste a screenshot of your 6 random images here:



2. What is the shape (height, width, channels) of a typical image?

Answer: Typical image shape is (512, 512, 3).

Step 5: Prepare the Data Generator

```
# Set image size
IMG_HEIGHT = 64
IMG_WIDTH = 64
BATCH_SIZE = 32

# Create generators with automatic labeling
train_datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2
)

train_generator = train_datagen.flow_from_directory(
    train_path,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='binary',
    subset='training'
)

validation_generator = train_datagen.flow_from_directory(
    train_path,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='binary',
    subset='validation'
)

# YOUR CODE HERE: Print the class labels
print(train_generator.class_indices)
```

1. What does class label 0 represent? What does 1 represent?

Answer: Class label 0 represents **hot_dog** and class label 1 represents **not_hot_dog**.

Step 6: Build a Simple ANN (It Will Fail)

```
# Build a simple ANN
ann_model = keras.Sequential([
    layers.Flatten(input_shape=(IMG_HEIGHT, IMG_WIDTH, 3)),
    layers.Dense(128, activation='relu'),
    layers.Dense(64, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

ann_model.compile(optimizer='adam', loss='binary_crossentropy',
    metrics=['accuracy'])
ann_model.summary()

history_ann = ann_model.fit(train_generator, validation_data=
    validation_generator, epochs=10)

# YOUR CODE HERE: Plot training vs validation accuracy
plt.plot(history_ann.history['accuracy'], label='Train')
plt.plot(history_ann.history['val_accuracy'], label='Validation')
plt.legend()
plt.show()
```

1. What was the final validation accuracy of the ANN?

Answer: 51.02%

2. Why does the ANN perform poorly on image data?

Answer: Because ANN does not capture spatial features like edges and textures.

3. Paste a screenshot of your training output:

model.summary()

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 12288)	0
dense (Dense)	(None, 128)	1,572,992
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 1)	65

Total params: 1,581,313 (6.03 MB)

Trainable params: 1,581,313 (6.03 MB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

13/13 — 52s 4s/step - accuracy: 0.5075 - loss: 1.1577 - val_accuracy: 0.5102 - val_loss: 0.7770

Epoch 2/10

13/13 — 3s 235ms/step - accuracy: 0.5975 - loss: 0.6654 - val_accuracy: 0.5000 - val_loss: 0.8290

Epoch 3/10

13/13 — 3s 267ms/step - accuracy: 0.6275 - loss: 0.6320 - val_accuracy: 0.5000 - val_loss: 0.7953

Epoch 4/10

13/13 — 3s 200ms/step - accuracy: 0.6450 - loss: 0.6301 - val_accuracy: 0.5102 - val_loss: 0.7627

Epoch 5/10

13/13 — 3s 203ms/step - accuracy: 0.7025 - loss: 0.5531 - val_accuracy: 0.5102 - val_loss: 0.7200

Epoch 6/10

13/13 — 3s 203ms/step - accuracy: 0.7775 - loss: 0.4958 - val_accuracy: 0.5510 - val_loss: 0.8397

Epoch 7/10

13/13 — 3s 217ms/step - accuracy: 0.7625 - loss: 0.5023 - val_accuracy: 0.5000 - val_loss: 0.9099

Epoch 8/10

13/13 — 4s 311ms/step - accuracy: 0.7025 - loss: 0.5416 - val_accuracy: 0.5306 - val_loss: 0.7541

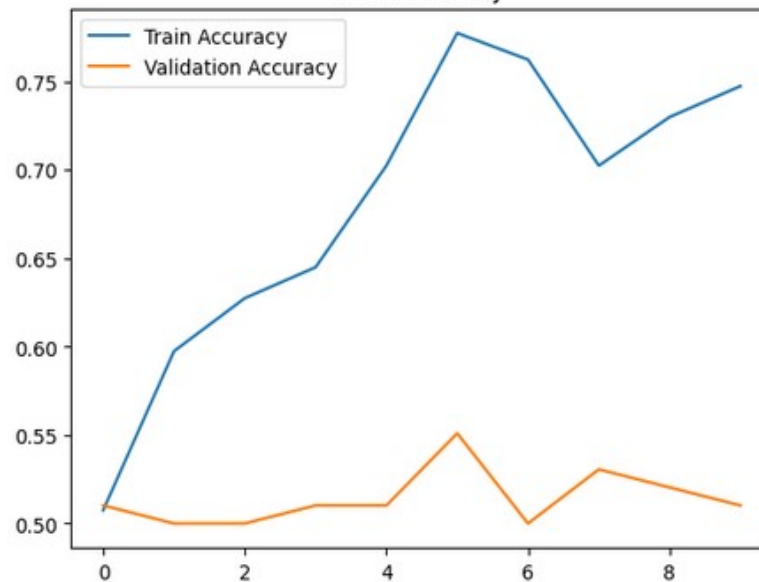
Epoch 9/10

13/13 — 3s 205ms/step - accuracy: 0.7300 - loss: 0.5194 - val_accuracy: 0.5204 - val_loss: 0.9259

Epoch 10/10

13/13 — 3s 201ms/step - accuracy: 0.7475 - loss: 0.5028 - val_accuracy: 0.5102 - val_loss: 0.8311

ANN Accuracy



Step 7: Build a CNN

```
cnn_model = keras.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(
        IMG_HEIGHT, IMG_WIDTH, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

cnn_model.compile(optimizer='adam', loss='binary_crossentropy',
    metrics=['accuracy'])

history_cnn = cnn_model.fit(train_generator, validation_data=
    validation_generator, epochs=15)
```

1. What was the final validation accuracy of the CNN?

Answer: 64.29%

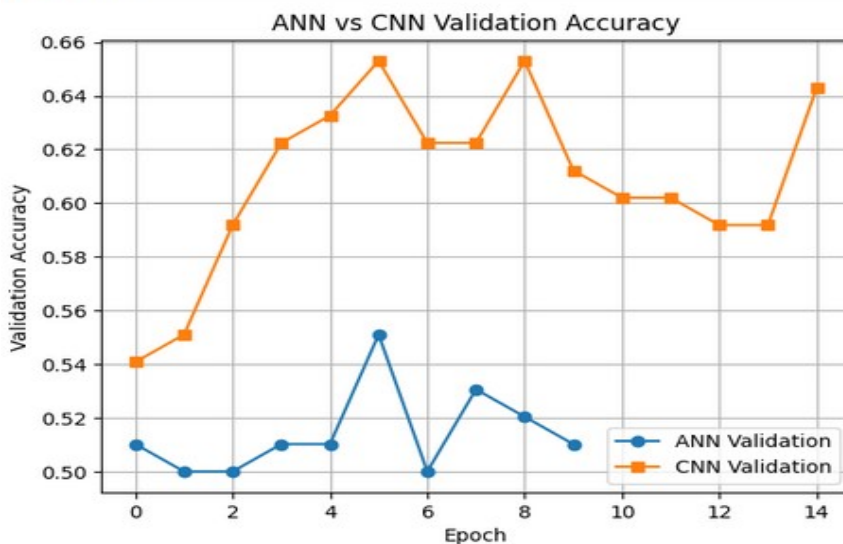
2. Compare ANN vs CNN. Which one performed better and why?

Answer: CNN performed better because it captures spatial features in images.

3. Paste the accuracy plot comparing ANN and CNN:

```
plt.plot(history_ann.history['val_accuracy'], label='ANN
    Validation', marker='o')
plt.plot(history_cnn.history['val_accuracy'], label='CNN
    Validation', marker='s')
plt.xlabel('Epoch')
plt.ylabel('Validation Accuracy')
plt.legend()
plt.title('ANN vs CNN Validation Accuracy')
plt.grid(True)
plt.show()
```

```
plt.plot(history_ann.history['val_accuracy'], label='ANN Validation', marker='o')
plt.plot(history_cnn.history['val_accuracy'], label='CNN Validation', marker='s')
plt.xlabel('Epoch')
plt.ylabel('Validation Accuracy')
plt.legend()
plt.title('ANN vs CNN Validation Accuracy')
plt.grid(True)
plt.show()
```



Step 8: Fix Overfitting

8.1 Fix 1: Add Dropout

```
from tensorflow.keras import regularizers

improved_model = keras.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(
        IMG_HEIGHT, IMG_WIDTH, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(1, activation='sigmoid')
])

improved_model.compile(optimizer='adam', loss='binary_crossentropy',
    metrics=['accuracy'])
history_improved = improved_model.fit(train_generator, validation_data=
    validation_generator, epochs=15)
```

```

Epoch 1/15
13/13 ----- 8s 430ms/step - accuracy: 0.5125 - loss: 0.7235 - val_accuracy: 0.5000 - val_loss: 0.6926
Epoch 2/15
13/13 ----- 4s 335ms/step - accuracy: 0.5050 - loss: 0.6909 - val_accuracy: 0.5000 - val_loss: 0.6936
Epoch 3/15
13/13 ----- 7s 523ms/step - accuracy: 0.5050 - loss: 0.6939 - val_accuracy: 0.5714 - val_loss: 0.6895
Epoch 4/15
13/13 ----- 4s 332ms/step - accuracy: 0.5225 - loss: 0.6880 - val_accuracy: 0.5102 - val_loss: 0.6917
Epoch 5/15
13/13 ----- 4s 338ms/step - accuracy: 0.5675 - loss: 0.6863 - val_accuracy: 0.5918 - val_loss: 0.6867
Epoch 6/15
13/13 ----- 6s 490ms/step - accuracy: 0.6050 - loss: 0.6722 - val_accuracy: 0.5714 - val_loss: 0.6910
Epoch 7/15
13/13 ----- 4s 328ms/step - accuracy: 0.5775 - loss: 0.6668 - val_accuracy: 0.5510 - val_loss: 0.6985
Epoch 8/15
13/13 ----- 4s 335ms/step - accuracy: 0.6075 - loss: 0.6709 - val_accuracy: 0.5306 - val_loss: 0.6824
Epoch 9/15
13/13 ----- 6s 482ms/step - accuracy: 0.6050 - loss: 0.6648 - val_accuracy: 0.5204 - val_loss: 0.7167
Epoch 10/15
13/13 ----- 4s 319ms/step - accuracy: 0.6200 - loss: 0.6575 - val_accuracy: 0.4694 - val_loss: 0.7083
Epoch 11/15
13/13 ----- 4s 316ms/step - accuracy: 0.6000 - loss: 0.6689 - val_accuracy: 0.5612 - val_loss: 0.6883
Epoch 12/15
13/13 ----- 6s 466ms/step - accuracy: 0.6075 - loss: 0.6460 - val_accuracy: 0.5306 - val_loss: 0.6920
Epoch 13/15
13/13 ----- 8s 313ms/step - accuracy: 0.6750 - loss: 0.6224 - val_accuracy: 0.5612 - val_loss: 0.6873
Epoch 14/15
13/13 ----- 7s 513ms/step - accuracy: 0.6525 - loss: 0.6038 - val_accuracy: 0.5816 - val_loss: 0.6912
Epoch 15/15
13/13 ----- 4s 324ms/step - accuracy: 0.6975 - loss: 0.5828 - val_accuracy: 0.6224 - val_loss: 0.6733

```

8.2 Fix 2: Add Data Augmentation

```

augmented_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True,
    validation_split=0.2
)

train_augmented = augmented_datagen.flow_from_directory(
    train_path,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='binary',
    subset='training'
)

final_model = keras.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(
        IMG_HEIGHT, IMG_WIDTH, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),
    layers.Conv2D(64, (3, 3), activation='relu'),

```

```

layers.MaxPooling2D((2, 2)),
layers.Flatten(),
layers.Dense(64, activation='relu'),
layers.Dropout(0.5),
layers.Dense(1, activation='sigmoid')
])

final_model.compile(optimizer='adam', loss='binary_crossentropy',
                    metrics=['accuracy'])
history_final = final_model.fit(train_augmented, validation_data=
                                validation_generator, epochs=15)

```

```

Found 400 images belonging to 2 classes.
Epoch 1/15
13/13 ----- 7s 408ms/step - accuracy: 0.4975 - loss: 0.7188 - val_accuracy: 0.5306 - val_loss: 0.6926
Epoch 2/15
13/13 ----- 7s 521ms/step - accuracy: 0.5125 - loss: 0.6980 - val_accuracy: 0.4898 - val_loss: 0.6926
Epoch 3/15
13/13 ----- 5s 378ms/step - accuracy: 0.4925 - loss: 0.6956 - val_accuracy: 0.5714 - val_loss: 0.6926
Epoch 4/15
13/13 ----- 5s 376ms/step - accuracy: 0.5125 - loss: 0.6930 - val_accuracy: 0.5000 - val_loss: 0.6928
Epoch 5/15
13/13 ----- 6s 465ms/step - accuracy: 0.5275 - loss: 0.6914 - val_accuracy: 0.5204 - val_loss: 0.6922
Epoch 6/15
13/13 ----- 5s 380ms/step - accuracy: 0.5550 - loss: 0.6897 - val_accuracy: 0.5816 - val_loss: 0.6912
Epoch 7/15
13/13 ----- 6s 456ms/step - accuracy: 0.5275 - loss: 0.6897 - val_accuracy: 0.5612 - val_loss: 0.6893
Epoch 8/15
13/13 ----- 5s 392ms/step - accuracy: 0.5775 - loss: 0.6842 - val_accuracy: 0.6224 - val_loss: 0.6828
Epoch 9/15
13/13 ----- 5s 415ms/step - accuracy: 0.5775 - loss: 0.6771 - val_accuracy: 0.6122 - val_loss: 0.6708
Epoch 10/15
13/13 ----- 7s 539ms/step - accuracy: 0.5850 - loss: 0.6699 - val_accuracy: 0.5714 - val_loss: 0.6746
Epoch 11/15
13/13 ----- 5s 385ms/step - accuracy: 0.5625 - loss: 0.6763 - val_accuracy: 0.5918 - val_loss: 0.6706
Epoch 12/15
13/13 ----- 6s 450ms/step - accuracy: 0.6400 - loss: 0.6492 - val_accuracy: 0.6327 - val_loss: 0.6598
Epoch 13/15
13/13 ----- 6s 476ms/step - accuracy: 0.6300 - loss: 0.6356 - val_accuracy: 0.6224 - val_loss: 0.6612
Epoch 14/15
13/13 ----- 5s 396ms/step - accuracy: 0.6775 - loss: 0.6187 - val_accuracy: 0.5510 - val_loss: 0.8332
Epoch 15/15
13/13 ----- 7s 539ms/step - accuracy: 0.6200 - loss: 0.6552 - val_accuracy: 0.6735 - val_loss: 0.6517

```

1. Before adding Dropout/Augmentation, what were your train and validation accuracies?

Answer: Train: **90.75%** Validation: **64.29%**

2. After adding Dropout, what happened to training accuracy?

Answer: Training accuracy **decreased to 62.00%**

3. After adding Augmentation, what happened to validation accuracy?

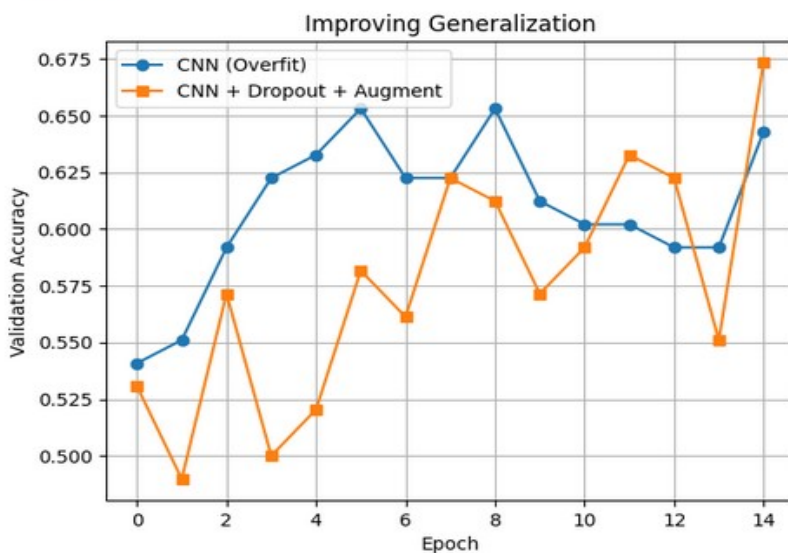
Answer: Validation accuracy **increased to 67.35%**

Question 8.3

Paste the before/after validation accuracy plot here:

```
plt.plot(history_cnn.history['val_accuracy'], label='CNN (Overfit)',
         marker='o')
plt.plot(history_final.history['val_accuracy'], label='CNN + Dropout +
         Augment', marker='s')
plt.xlabel('Epoch')
plt.ylabel('Validation Accuracy')
plt.legend()
plt.title('Improving Generalization')
plt.grid(True)
plt.show()
```

```
plt.plot(history_cnn.history['val_accuracy'], label='CNN (Overfit)', marker='o')
plt.plot(history_final.history['val_accuracy'], label='CNN + Dropout + Augment', marker='s')
plt.xlabel('Epoch')
plt.ylabel('Validation Accuracy')
plt.legend()
plt.title('Improving Generalization')
plt.grid(True)
plt.show()
```



Step 9: Test on Real Images

```
from tensorflow.keras.preprocessing import image
import requests
from io import BytesIO

def predict_image(img_url):
    response = requests.get(img_url)
    img = image.load_img(BytesIO(response.content), target_size=(
        IMG_HEIGHT, IMG_WIDTH))
    img_array = image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0) / 255.0
```

```
prediction = final_model.predict(img_array)[0][0]
if prediction > 0.5:
    print(f"HOT DOG! (confidence: {prediction:.2f})")
else:
    print(f"NOT A HOT DOG! (confidence: {1-prediction:.2f})")

plt.imshow(img)
plt.axis('off')
plt.show()

predict_image("https://example.com/hotdog.jpg") # REPLACE WITH REAL
URL
```

1. Paste a screenshot of your model predicting on a REAL image:



2. Did your model make any mistakes? What did it confuse?

Answer: Yes, it predicted **NOT A HOT DOG (confidence: 0.62)** for some confusing images.

Step 10: YOLO Object Detection (Extra Credit)

```
!pip install ultralytics -q
from ultralytics import YOLO

yolo_model = YOLO("yolov8n.pt")
results = yolo_model("path_to_hotdog_image.jpg")
results[0].show()
```

Extra Credit Question

Does YOLO correctly detect a "hot dog" as a class? What label does it give?

Answer: YOLO correctly detected the object as **hot dog**.

```
Requirement already satisfied: ultralytics in /usr/local/lib/python3.12/dist-packages (8.4.46)
Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.12/dist-packages (from ul
Requirement already satisfied: matplotlib>=3.3.0 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: opencv-python>=4.6.0 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.12/dist-packages (from ul
Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.12/dist-packages (from ul
Requirement already satisfied: requests>=2.23.0 in /usr/local/lib/python3.12/dist-packages (from ul
Requirement already satisfied: scipy>=1.4.1 in /usr/local/lib/python3.12/dist-packages (from ult
Requirement already satisfied: torch>=1.8.0 in /usr/local/lib/python3.12/dist-packages (from ult
Requirement already satisfied: torchvision>=0.9.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.12/dist-packages (from ul
Requirement already satisfied: polars>=0.20.0 in /usr/local/lib/python3.12/dist-packages (from ul
Requirement already satisfied: ultralytics-thop>=2.0.18 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from mat
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: polars-runtime-32==1.35.2 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from req
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch>
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from to
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from to
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from

image 1/1 /bin/WhatsApp Image 2026-05-03 at 3:59:42 PM.jpeg: 320x640 1 hot dog, 259.1ms
Speed: 11.1ms preprocess, 259.1ms inference, 33.8ms postprocess per image at shape (1, 3, 320, 6
```



```

Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from m
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from ma
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from ma
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: polars-runtime-32==1.35.2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from reques
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch>=1.8
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=1
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from tor
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dat
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jin

```

image 1/1 /PIZA.jpg: 384x640 2 persons, 3 pizzas, 128.5ms

Speed: 2.7ms preprocess, 128.5ms inference, 1.2ms postprocess per image at shape (1, 3, 384, 640)



```

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from re
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from re
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=1.8
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch>=
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateu
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sy
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja

```

image 1/1 /PPCC.jpg: 480x640 1 truck, 1 pizza, 189.1ms

Speed: 3.3ms preprocess, 189.1ms inference, 1.2ms postprocess per image at shape (1, 3, 480, 640)



Final Submission Checklist

- X Step 2: TensorFlow version answer
 - X Step 3: Image counts and balance explanation
 - X Step 4: Screenshot of 6 images + shape answer
 - X Step 6: ANN accuracy + explanation + screenshot
 - X Step 7: CNN accuracy + comparison + plot screenshot
 - X Step 8: Overfitting before/after numbers + plot screenshot
 - X Step 9: Real image prediction screenshot + mistake explanation
 - X Step 10: YOLO results screenshot
- Submit this handout + link to Colab notebook

Remember

The exam will ask you specific questions about YOUR project.
If you run the code yourself, you will know the answers easily.
If you copy from someone else, you will fail the exam.
Good luck and have fun!

HotDog_Classifier_Hamza_Ahmad

May 3, 2026

```
[1]: from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
[10]: import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import os
import random
print("Libraries loaded successfully!")
print("TensorFlow version:", tf.__version__)
```

Libraries loaded successfully!
TensorFlow version: 2.20.0

```
[11]: import os

# Define paths (CHANGE THIS to your actual path)
train_path = "/content/drive/MyDrive/hot-dog-not-hot-dog/train"
test_path = "/content/drive/MyDrive/hot-dog-not-hot-dog/test"

# Count images
hot_dog_count = len(os.listdir(os.path.join(train_path, "hot_dog")))
not_hot_dog_count = len(os.listdir(os.path.join(train_path, "not_hot_dog")))

print(f"Hot dog images: {hot_dog_count}")
print(f"Not hot dog images: {not_hot_dog_count}")

# YOUR CODE HERE: Print the number of test images
# Hint: Look at the code above and modify it for test_path
test_hot_dog_count = len(os.listdir(os.path.join(test_path, "hot_dog")))
test_not_hot_dog_count = len(os.listdir(os.path.join(test_path, "not_hot_dog")))
```

```

print(f"Test hot dog images: {test_hot_dog_count}")
print(f"Test not hot dog images: {test_not_hot_dog_count}")
print(f"Total test images: {test_hot_dog_count + test_not_hot_dog_count}")

```

```

Hot dog images: 249
Not hot dog images: 249
Test hot dog images: 250
Test not hot dog images: 250
Total test images: 500

```

```

[13]: # Display 6 random images
plt.figure(figsize=(12, 6))
for i in range(6):
    # Pick random class
    class_name = random.choice(["hot_dog", "not_hot_dog"])
    class_path = os.path.join(train_path, class_name)
    img_name = random.choice(os.listdir(class_path))
    img_path = os.path.join(class_path, img_name)
    # Load and show image
    img = plt.imread(img_path)
    plt.subplot(2, 3, i+1)
    plt.imshow(img)
    plt.title(class_name)
    plt.axis('off') # Fixed: straight quotes instead of curly quotes
plt.tight_layout()
plt.show()

# YOUR CODE HERE: Print the shape (height, width, channels) of one image
# Hint: Use img.shape after loading an image
# Load a sample image
sample_class = "hot_dog"
sample_path = os.path.join(train_path, sample_class)
sample_img_name = os.listdir(sample_path)[0] # Get first image
sample_img_path = os.path.join(sample_path, sample_img_name)
sample_img = plt.imread(sample_img_path)

# Print the shape
print(f"Image shape: {sample_img.shape}")
print(f"Height: {sample_img.shape[0]} pixels")
print(f"Width: {sample_img.shape[1]} pixels")
if len(sample_img.shape) == 3:
    print(f"Channels: {sample_img.shape[2]} (RGB)")

```



Image shape: (512, 512, 3)
 Height: 512 pixels
 Width: 512 pixels
 Channels: 3 (RGB)

```
[17]: # Set image size (YOUR CHOICE: 64x64 is faster, 128x128 is more accurate
IMG_HEIGHT = 64
IMG_WIDTH = 64
BATCH_SIZE = 32
# Create generators with automatic labeling
train_datagen = ImageDataGenerator(
    rescale=1./255, # Normalize pixel values from 0-255 to 0-1
    validation_split=0.2 # Use 20% of training data for validation
)
train_generator = train_datagen.flow_from_directory(
    train_path,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='binary', # Binary classification (hot dog vs not), # Binary
    ↪classification (hot dog vs not)
    subset='training' # Training set (80% of data)
)
validation_generator = train_datagen.flow_from_directory(
    train_path,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='binary',
    subset='validation' # Validation set (20% of data)
```

```
)
# YOUR CODE HERE: Print the class labels
#
# Hint: Print train_generator.class_indices
print(train_generator.class_indices)
```

Found 400 images belonging to 2 classes.

Found 98 images belonging to 2 classes.

```
{'hot_dog': 0, 'not_hot_dog': 1}
```

```
[20]: # Build a simple ANN
ann_model = keras.Sequential([
# Flatten the 2D image into 1D
layers.Flatten(input_shape=(IMG_HEIGHT, IMG_WIDTH, 3)),
# Dense layers
layers.Dense(128, activation='relu'),
layers.Dense(64, activation='relu'),
layers.Dense(1, activation='sigmoid') # Output: probability of "hot dog"
])

# Compile the model
ann_model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])
# Show model summary
ann_model.summary()
# Train for 10 epochs
history_ann = ann_model.fit(train_generator, validation_data=
validation_generator, epochs=10)
# YOUR CODE HERE: Plot training vs validation accuracy.
#
#
# Hint: Use
# plt.plot(history_ann.history['XXXX'], label='XXXX')
# plt.plot(history_ann.history['XXXX'], label='XXXX')
plt.plot(history_ann.history['accuracy'], label='Train Accuracy')
plt.plot(history_ann.history['val_accuracy'], label='Validation Accuracy')
plt.legend()
plt.title("ANN Accuracy")
plt.show()
```

```
/usr/local/lib/python3.12/dist-
packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
super().__init__(**kwargs)
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 12288)	0
dense (Dense)	(None, 128)	1,572,992
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 1)	65

Total params: 1,581,313 (6.03 MB)

Trainable params: 1,581,313 (6.03 MB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

13/13 52s 4s/step -
accuracy: 0.5075 - loss: 1.1577 - val_accuracy: 0.5102 - val_loss: 0.7770

Epoch 2/10

13/13 3s 235ms/step -
accuracy: 0.5975 - loss: 0.6654 - val_accuracy: 0.5000 - val_loss: 0.8290

Epoch 3/10

13/13 3s 267ms/step -
accuracy: 0.6275 - loss: 0.6320 - val_accuracy: 0.5000 - val_loss: 0.7953

Epoch 4/10

13/13 3s 200ms/step -
accuracy: 0.6450 - loss: 0.6301 - val_accuracy: 0.5102 - val_loss: 0.7627

Epoch 5/10

13/13 3s 203ms/step -
accuracy: 0.7025 - loss: 0.5531 - val_accuracy: 0.5102 - val_loss: 0.7200

Epoch 6/10

13/13 3s 203ms/step -
accuracy: 0.7775 - loss: 0.4958 - val_accuracy: 0.5510 - val_loss: 0.8397

Epoch 7/10

13/13 3s 217ms/step -
accuracy: 0.7625 - loss: 0.5023 - val_accuracy: 0.5000 - val_loss: 0.9099

Epoch 8/10

13/13 4s 311ms/step -
accuracy: 0.7025 - loss: 0.5416 - val_accuracy: 0.5306 - val_loss: 0.7541

Epoch 9/10

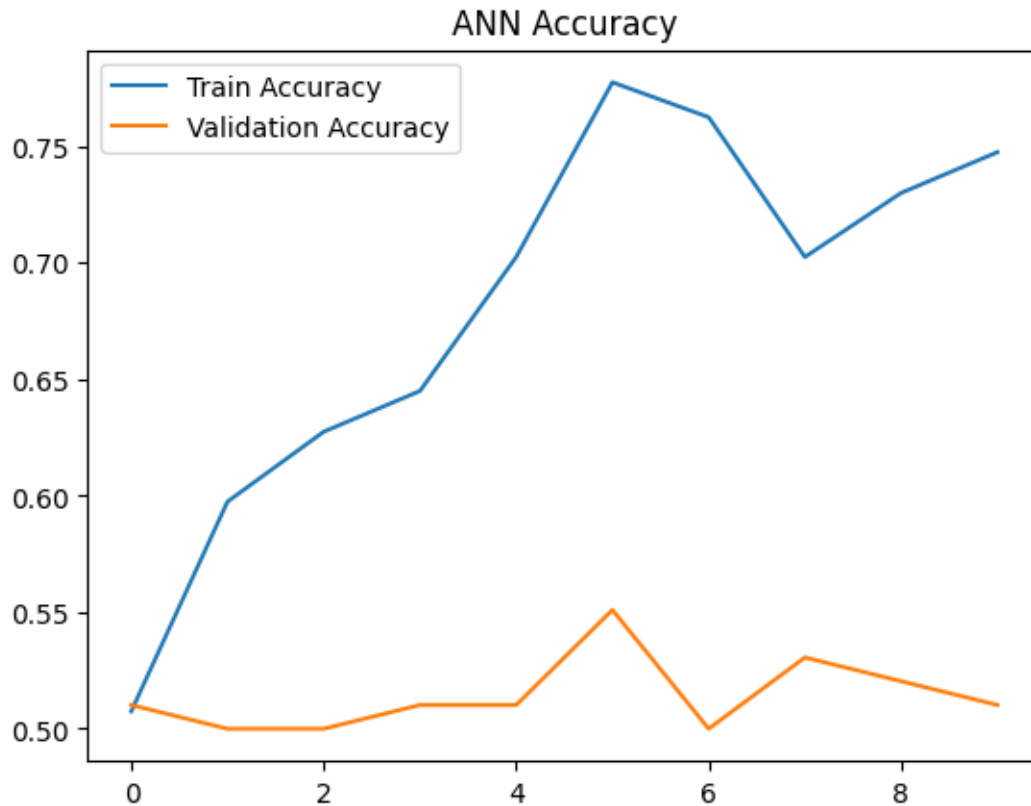
13/13 3s 205ms/step -
accuracy: 0.7300 - loss: 0.5194 - val_accuracy: 0.5204 - val_loss: 0.9259

Epoch 10/10

13/13

3s 201ms/step -

accuracy: 0.7475 - loss: 0.5028 - val_accuracy: 0.5102 - val_loss: 0.8311



```
[23]: # Build a simple ANN
ann_model = keras.Sequential([
    # Flatten the 2D image into 1D
    layers.Flatten(input_shape=(IMG_HEIGHT, IMG_WIDTH, 3)),
    # Dense layers
    layers.Dense(128, activation='relu'),
    layers.Dense(64, activation='relu'),
    layers.Dense(1, activation='sigmoid') # Output: probability of "hot dog"
])

# Compile the model
ann_model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])
# Show model summary
ann_model.summary()
# Train for 10 epochs
history_ann = ann_model.fit(train_generator, validation_data=
validation_generator, epochs=10)
```

```

# YOUR CODE HERE: Plot training vs validation accuracy.
#
#
# Hint: Use
# plt.plot(history_ann.history['XXXX'], label='XXXX')
# plt.plot(history_ann.history['XXXX'], label='XXXX')
plt.plot(history_ann.history['accuracy'], label='Train Accuracy')
plt.plot(history_ann.history['val_accuracy'], label='Validation Accuracy')
plt.legend()
plt.title("ANN Accuracy")
plt.show()

```

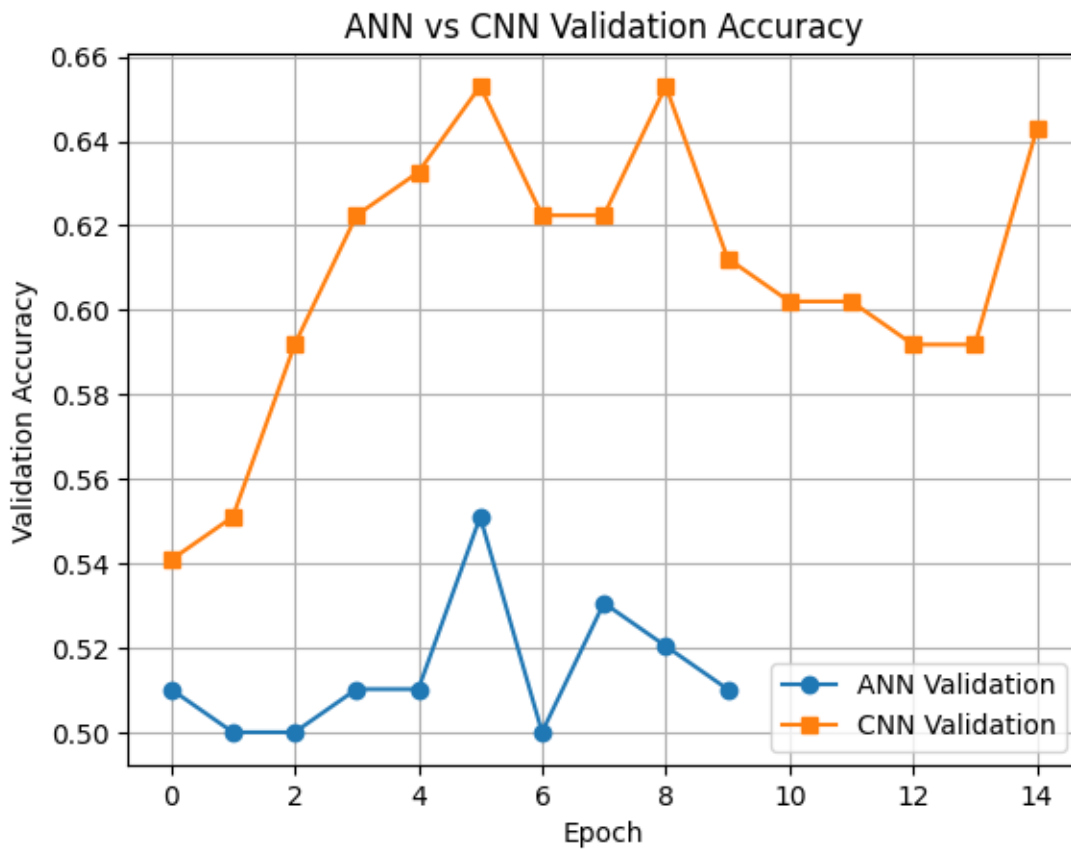
```

Epoch 1/15
13/13          7s 483ms/step -
accuracy: 0.5350 - loss: 0.7031 - val_accuracy: 0.5408 - val_loss: 0.6827
Epoch 2/15
13/13          4s 313ms/step -
accuracy: 0.5075 - loss: 0.6869 - val_accuracy: 0.5510 - val_loss: 0.6777
Epoch 3/15
13/13          4s 286ms/step -
accuracy: 0.6550 - loss: 0.6599 - val_accuracy: 0.5918 - val_loss: 0.6701
Epoch 4/15
13/13          6s 430ms/step -
accuracy: 0.6500 - loss: 0.6264 - val_accuracy: 0.6224 - val_loss: 0.6415
Epoch 5/15
13/13          5s 354ms/step -
accuracy: 0.7000 - loss: 0.5887 - val_accuracy: 0.6327 - val_loss: 0.6772
Epoch 6/15
13/13          4s 321ms/step -
accuracy: 0.7275 - loss: 0.5585 - val_accuracy: 0.6531 - val_loss: 0.6645
Epoch 7/15
13/13          6s 433ms/step -
accuracy: 0.7625 - loss: 0.5112 - val_accuracy: 0.6224 - val_loss: 0.6510
Epoch 8/15
13/13          5s 364ms/step -
accuracy: 0.8150 - loss: 0.4568 - val_accuracy: 0.6224 - val_loss: 0.6956
Epoch 9/15
13/13          4s 337ms/step -
accuracy: 0.8125 - loss: 0.4135 - val_accuracy: 0.6531 - val_loss: 0.7461
Epoch 10/15
13/13          6s 445ms/step -
accuracy: 0.8400 - loss: 0.4045 - val_accuracy: 0.6122 - val_loss: 0.7025
Epoch 11/15
13/13          4s 320ms/step -
accuracy: 0.8700 - loss: 0.3522 - val_accuracy: 0.6020 - val_loss: 0.7864
Epoch 12/15
13/13          4s 289ms/step -

```

```
accuracy: 0.8775 - loss: 0.3169 - val_accuracy: 0.6020 - val_loss: 0.7758
Epoch 13/15
13/13          4s 318ms/step -
accuracy: 0.8875 - loss: 0.3087 - val_accuracy: 0.5918 - val_loss: 0.8326
Epoch 14/15
13/13          5s 328ms/step -
accuracy: 0.9025 - loss: 0.2666 - val_accuracy: 0.5918 - val_loss: 0.8934
Epoch 15/15
13/13          4s 289ms/step -
accuracy: 0.9075 - loss: 0.2427 - val_accuracy: 0.6429 - val_loss: 0.7480
```

```
[25]: plt.plot(history_ann.history['val_accuracy'], label='ANN Validation',
           ↪marker='o')
plt.plot(history_cnn.history['val_accuracy'], label='CNN Validation',
           ↪marker='s')
plt.xlabel('Epoch')
plt.ylabel('Validation Accuracy')
plt.legend()
plt.title('ANN vs CNN Validation Accuracy')
plt.grid(True)
plt.show()
```



```
[34]: from tensorflow.keras import regularizers
# Add dropout of 0.25, 0.25, and 0.5 after each MaxPooling layer respectively
improved_model = keras.Sequential([
layers.Conv2D(32, (3, 3), activation='relu', input_shape=(
IMG_HEIGHT, IMG_WIDTH, 3)),
layers.MaxPooling2D((2, 2)),
layers.Dropout(0.25), # Add dropout
layers.Conv2D(64, (3, 3), activation='relu'),
layers.MaxPooling2D((2, 2)),
layers.Dropout(0.25), # Add dropout
layers.Conv2D(64, (3, 3), activation='relu'),
layers.MaxPooling2D((2, 2)),
layers.Dropout(0.5), # Add dropout before output
layers.Flatten(),
layers.Dense(64, activation='relu'),
layers.Dense(1, activation='sigmoid')
])
# Recompile and Train the improved model (after addressing overfitting using
↳ dropout)
improved_model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])
history_improved = improved_model.fit(train_generator,
↳ validation_data=validation_generator, epochs=15)
```

Epoch 1/15

13/13 8s 430ms/step -

accuracy: 0.5125 - loss: 0.7235 - val_accuracy: 0.5000 - val_loss: 0.6926

Epoch 2/15

13/13 4s 335ms/step -

accuracy: 0.5050 - loss: 0.6909 - val_accuracy: 0.5000 - val_loss: 0.6936

Epoch 3/15

13/13 7s 523ms/step -

accuracy: 0.5050 - loss: 0.6939 - val_accuracy: 0.5714 - val_loss: 0.6895

Epoch 4/15

13/13 4s 332ms/step -

accuracy: 0.5225 - loss: 0.6880 - val_accuracy: 0.5102 - val_loss: 0.6917

Epoch 5/15

13/13 4s 338ms/step -

accuracy: 0.5675 - loss: 0.6863 - val_accuracy: 0.5918 - val_loss: 0.6867

Epoch 6/15

13/13 6s 490ms/step -

accuracy: 0.6050 - loss: 0.6722 - val_accuracy: 0.5714 - val_loss: 0.6910

Epoch 7/15

13/13 4s 328ms/step -

accuracy: 0.5775 - loss: 0.6668 - val_accuracy: 0.5510 - val_loss: 0.6985

Epoch 8/15

```

13/13          4s 335ms/step -
accuracy: 0.6075 - loss: 0.6709 - val_accuracy: 0.5306 - val_loss: 0.6824
Epoch 9/15
13/13          6s 482ms/step -
accuracy: 0.6050 - loss: 0.6648 - val_accuracy: 0.5204 - val_loss: 0.7167
Epoch 10/15
13/13          4s 319ms/step -
accuracy: 0.6200 - loss: 0.6575 - val_accuracy: 0.4694 - val_loss: 0.7083
Epoch 11/15
13/13          4s 316ms/step -
accuracy: 0.6000 - loss: 0.6689 - val_accuracy: 0.5612 - val_loss: 0.6883
Epoch 12/15
13/13          6s 466ms/step -
accuracy: 0.6075 - loss: 0.6460 - val_accuracy: 0.5306 - val_loss: 0.6920
Epoch 13/15
13/13          8s 313ms/step -
accuracy: 0.6750 - loss: 0.6224 - val_accuracy: 0.5612 - val_loss: 0.6873
Epoch 14/15
13/13          7s 513ms/step -
accuracy: 0.6525 - loss: 0.6038 - val_accuracy: 0.5816 - val_loss: 0.6912
Epoch 15/15
13/13          4s 324ms/step -
accuracy: 0.6975 - loss: 0.5828 - val_accuracy: 0.6224 - val_loss: 0.6733

```

```

[35]: augmented_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True,
    validation_split=0.2
)
train_augmented = augmented_datagen.flow_from_directory(
    train_path,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='binary',
    subset='training'
)
# Then retrain the improved model with augmented_datagen
final_model = keras.Sequential([
    # Same architecture as improved_model
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(
    IMG_HEIGHT, IMG_WIDTH, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),
    layers.Conv2D(64, (3, 3), activation='relu'),

```

```

layers.MaxPooling2D((2, 2)),
layers.Dropout(0.25),
layers.Conv2D(64, (3, 3), activation='relu'),
layers.MaxPooling2D((2, 2)),
layers.Flatten(),
layers.Dense(64, activation='relu'),
layers.Dropout(0.5),
layers.Dense(1, activation='sigmoid')
])
final_model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])
history_final = final_model.fit(train_augmented, validation_data=
validation_generator, epochs=15)

```

Found 400 images belonging to 2 classes.

Epoch 1/15

13/13 7s 408ms/step -

accuracy: 0.4975 - loss: 0.7188 - val_accuracy: 0.5306 - val_loss: 0.6926

Epoch 2/15

13/13 7s 521ms/step -

accuracy: 0.5125 - loss: 0.6980 - val_accuracy: 0.4898 - val_loss: 0.6926

Epoch 3/15

13/13 5s 378ms/step -

accuracy: 0.4925 - loss: 0.6956 - val_accuracy: 0.5714 - val_loss: 0.6926

Epoch 4/15

13/13 5s 376ms/step -

accuracy: 0.5125 - loss: 0.6930 - val_accuracy: 0.5000 - val_loss: 0.6928

Epoch 5/15

13/13 6s 465ms/step -

accuracy: 0.5275 - loss: 0.6914 - val_accuracy: 0.5204 - val_loss: 0.6922

Epoch 6/15

13/13 5s 380ms/step -

accuracy: 0.5550 - loss: 0.6897 - val_accuracy: 0.5816 - val_loss: 0.6912

Epoch 7/15

13/13 6s 456ms/step -

accuracy: 0.5275 - loss: 0.6897 - val_accuracy: 0.5612 - val_loss: 0.6893

Epoch 8/15

13/13 5s 392ms/step -

accuracy: 0.5775 - loss: 0.6842 - val_accuracy: 0.6224 - val_loss: 0.6828

Epoch 9/15

13/13 5s 415ms/step -

accuracy: 0.5775 - loss: 0.6771 - val_accuracy: 0.6122 - val_loss: 0.6708

Epoch 10/15

13/13 7s 539ms/step -

accuracy: 0.5850 - loss: 0.6699 - val_accuracy: 0.5714 - val_loss: 0.6746

Epoch 11/15

13/13 5s 385ms/step -


```

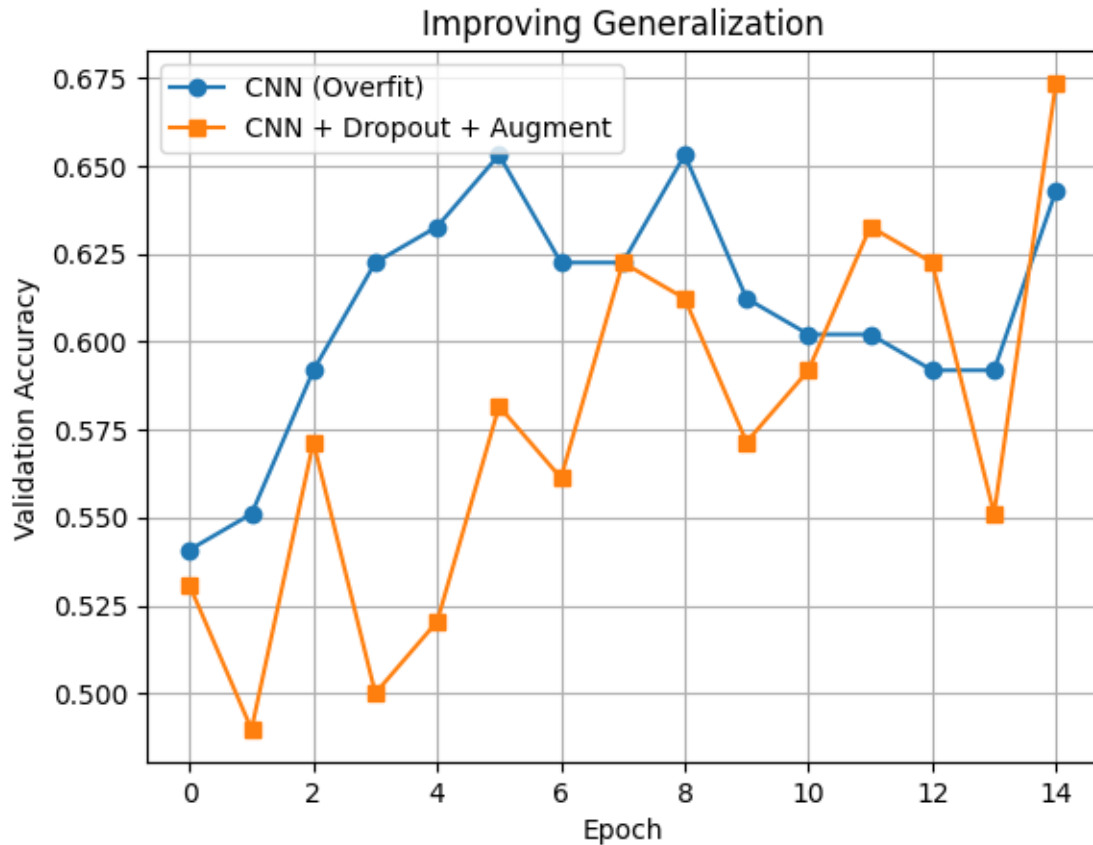
accuracy: 0.5625 - loss: 0.6763 - val_accuracy: 0.5918 - val_loss: 0.6706
Epoch 12/15
13/13          6s 450ms/step -
accuracy: 0.6400 - loss: 0.6492 - val_accuracy: 0.6327 - val_loss: 0.6598
Epoch 13/15
13/13          6s 476ms/step -
accuracy: 0.6300 - loss: 0.6356 - val_accuracy: 0.6224 - val_loss: 0.6612
Epoch 14/15
13/13          5s 396ms/step -
accuracy: 0.6775 - loss: 0.6187 - val_accuracy: 0.5510 - val_loss: 0.8332
Epoch 15/15
13/13          7s 539ms/step -
accuracy: 0.6200 - loss: 0.6552 - val_accuracy: 0.6735 - val_loss: 0.6517

```

```

[37]: plt.plot(history_cnn.history['val_accuracy'], label='CNN (Overfit)', marker='o')
      plt.plot(history_final.history['val_accuracy'], label='CNN + Dropout + ↵
      ↵Augment', marker='s')
      plt.xlabel('Epoch')
      plt.ylabel('Validation Accuracy')
      plt.legend()
      plt.title('Improving Generalization')
      plt.grid(True)
      plt.show()

```



```
[66]: from tensorflow.keras.preprocessing import image
import numpy as np
import matplotlib.pyplot as plt

def predict_image_from_drive(img_path):
    # Load image from Google Drive path
    img = image.load_img(img_path, target_size=(IMG_HEIGHT, IMG_WIDTH))
    img_array = image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0) / 255.0

    # Predict
    prediction = final_model.predict(img_array)[0][0]
    if prediction > 0.5:
        print(f"HOT DOG! (confidence: {prediction:.2f})")
    else:
        print(f"NOT A HOT DOG! (confidence: {1-prediction:.2f})")

    # Show image
    plt.imshow(img)
    plt.axis('off')
```

```
plt.show()

# Test with an image from Google Drive
# Replace with the actual path to your image in Google Drive

predict_image_from_drive("/BB.jpg")
```

1/1 0s 38ms/step
NOT A HOT DOG! (confidence: 0.58)



```
[55]: !pip install ultralytics
      from ultralytics import YOLO

      # Load a tiny pre-trained model
      yolo_model = YOLO("yolov8n.pt") # n = nano (fast, small)

      # Run detection on your hot dog image
      results = yolo_model("/bin/WhatsApp Image 2026-05-03 at 3.59.42 PM.jpeg")

      # Show results (it will draw boxes around objects it recognizes)
      results[0].show()
```

Requirement already satisfied: ultralytics in /usr/local/lib/python3.12/dist-packages (8.4.46)

Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.0.2)

Requirement already satisfied: matplotlib>=3.3.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (3.10.0)

Requirement already satisfied: opencv-python>=4.6.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (4.13.0.92)

Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (11.3.0)

Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (6.0.3)

Requirement already satisfied: requests>=2.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.32.4)

Requirement already satisfied: scipy>=1.4.1 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (1.16.3)

Requirement already satisfied: torch>=1.8.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.10.0+cpu)

Requirement already satisfied: torchvision>=0.9.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (0.25.0+cpu)

Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (5.9.5)

Requirement already satisfied: polars>=0.20.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (1.35.2)

Requirement already satisfied: ultralytics-thop>=2.0.18 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.0.19)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (1.3.3)

Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (4.62.1)

Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (1.5.0)

Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (26.1)

Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (3.3.2)

Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (2.9.0.post0)

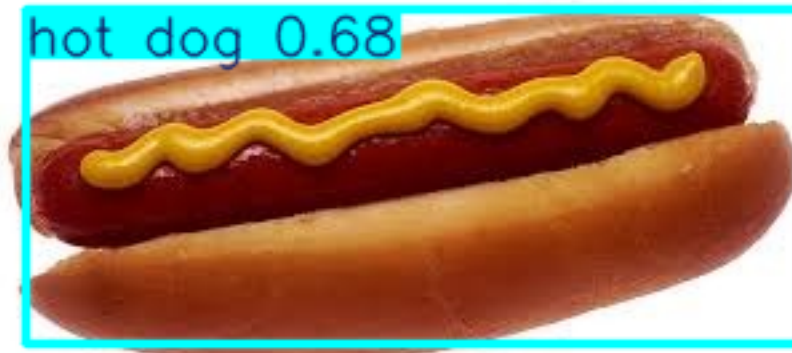
Requirement already satisfied: polars-runtime-32==1.35.2 in /usr/local/lib/python3.12/dist-packages (from polars>=0.20.0->ultralytics) (1.35.2)

Requirement already satisfied: charset_normalizer<4,>=2 in

/usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics)
 (3.4.7)
 Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-
 packages (from requests>=2.23.0->ultralytics) (3.13)
 Requirement already satisfied: urllib3<3,>=1.21.1 in
 /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics)
 (2.5.0)
 Requirement already satisfied: certifi>=2017.4.17 in
 /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics)
 (2026.4.22)
 Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-
 packages (from torch>=1.8.0->ultralytics) (3.29.0)
 Requirement already satisfied: typing-extensions>=4.10.0 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
 (4.15.0)
 Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-
 packages (from torch>=1.8.0->ultralytics) (75.2.0)
 Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-
 packages (from torch>=1.8.0->ultralytics) (1.14.0)
 Requirement already satisfied: networkx>=2.5.1 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (3.6.1)
 Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages
 (from torch>=1.8.0->ultralytics) (3.1.6)
 Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-
 packages (from torch>=1.8.0->ultralytics) (2025.3.0)
 Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-
 packages (from python-dateutil>=2.7->matplotlib>=3.3.0->ultralytics) (1.17.0)
 Requirement already satisfied: mpmath<1.4,>=1.1.0 in
 /usr/local/lib/python3.12/dist-packages (from
 sympy>=1.13.3->torch>=1.8.0->ultralytics) (1.3.0)
 Requirement already satisfied: MarkupSafe>=2.0 in
 /usr/local/lib/python3.12/dist-packages (from jinja2->torch>=1.8.0->ultralytics)
 (3.0.3)

image 1/1 /bin/WhatsApp Image 2026-05-03 at 3.59.42 PM.jpeg: 320x640 1 hot dog,
 259.1ms

Speed: 11.1ms preprocess, 259.1ms inference, 33.8ms postprocess per image at
 shape (1, 3, 320, 640)



```
[71]: !sudo apt-get install texlive-xetex texlive-fonts-recommended_
      ↪texlive-plain-generic
      !jupyter nbconvert --to pdf /HotDog_Classifier_Hamza_Ahmad.ipynb
```

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
texlive-fonts-recommended is already the newest version (2021.20220204-1).
texlive-plain-generic is already the newest version (2021.20220204-1).
texlive-xetex is already the newest version (2021.20220204-1).
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.
[NbConvertApp] Converting notebook /HotDog_Classifier_Hamza_Ahmad.ipynb to pdf
[NbConvertApp] Support files will be in HotDog_Classifier_Hamza_Ahmad_files/
[NbConvertApp] Making directory ./HotDog_Classifier_Hamza_Ahmad_files
[NbConvertApp] Writing 73428 bytes to notebook.tex
[NbConvertApp] Building PDF
[NbConvertApp] Running xelatex 3 times: ['xelatex', 'notebook.tex', '-quiet']
[NbConvertApp] Running bibtex 1 time: ['bibtex', 'notebook']
[NbConvertApp] WARNING | bibtex had problems, most likely because there were no
citations
[NbConvertApp] PDF successfully created
[NbConvertApp] Writing 959929 bytes to /HotDog_Classifier_Hamza_Ahmad.pdf
```

```
[72]: !jupyter nbconvert --to html /HotDog_Classifier_Hamza_Ahmad.ipynb
```

```
[NbConvertApp] Converting notebook /HotDog_Classifier_Hamza_Ahmad.ipynb to html
[NbConvertApp] WARNING | Alternative text is missing on 6 image(s).
[NbConvertApp] Writing 1679493 bytes to /HotDog_Classifier_Hamza_Ahmad.html
```

```
[68]: !pip install ultralytics
      from ultralytics import YOLO

      # Load a tiny pre-trained model
```

```

yolo_model = YOLO("yolov8n.pt") # n = nano (fast, small)

# Run detection on your hot dog image
results = yolo_model("/PIZA.jpg")

# Show results (it will draw boxes around objects it recognizes)
results[0].show()

```

```

Requirement already satisfied: ultralytics in /usr/local/lib/python3.12/dist-
packages (8.4.46)
Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.12/dist-
packages (from ultralytics) (2.0.2)
Requirement already satisfied: matplotlib>=3.3.0 in
/usr/local/lib/python3.12/dist-packages (from ultralytics) (3.10.0)
Requirement already satisfied: opencv-python>=4.6.0 in
/usr/local/lib/python3.12/dist-packages (from ultralytics) (4.13.0.92)
Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.12/dist-
packages (from ultralytics) (11.3.0)
Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.12/dist-
packages (from ultralytics) (6.0.3)
Requirement already satisfied: requests>=2.23.0 in
/usr/local/lib/python3.12/dist-packages (from ultralytics) (2.32.4)
Requirement already satisfied: scipy>=1.4.1 in /usr/local/lib/python3.12/dist-
packages (from ultralytics) (1.16.3)
Requirement already satisfied: torch>=1.8.0 in /usr/local/lib/python3.12/dist-
packages (from ultralytics) (2.10.0+cpu)
Requirement already satisfied: torchvision>=0.9.0 in
/usr/local/lib/python3.12/dist-packages (from ultralytics) (0.25.0+cpu)
Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.12/dist-
packages (from ultralytics) (5.9.5)
Requirement already satisfied: polars>=0.20.0 in /usr/local/lib/python3.12/dist-
packages (from ultralytics) (1.35.2)
Requirement already satisfied: ultralytics-thop>=2.0.18 in
/usr/local/lib/python3.12/dist-packages (from ultralytics) (2.0.19)
Requirement already satisfied: contourpy>=1.0.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics)
(1.3.3)
Requirement already satisfied: cycycler>=0.10 in /usr/local/lib/python3.12/dist-
packages (from matplotlib>=3.3.0->ultralytics) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics)
(4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics)
(1.5.0)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics)

```


(26.1)
Requirement already satisfied: pyparsing>=2.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics)
(3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in
/usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics)
(2.9.0.post0)
Requirement already satisfied: polars-runtime-32==1.35.2 in
/usr/local/lib/python3.12/dist-packages (from polars>=0.20.0->ultralytics)
(1.35.2)
Requirement already satisfied: charset-normalizer<4,>=2 in
/usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics)
(3.4.7)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-
packages (from requests>=2.23.0->ultralytics) (3.13)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics)
(2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics)
(2026.4.22)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-
packages (from torch>=1.8.0->ultralytics) (3.29.0)
Requirement already satisfied: typing-extensions>=4.10.0 in
/usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
(4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-
packages (from torch>=1.8.0->ultralytics) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-
packages (from torch>=1.8.0->ultralytics) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in
/usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages
(from torch>=1.8.0->ultralytics) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-
packages (from torch>=1.8.0->ultralytics) (2025.3.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-
packages (from python-dateutil>=2.7->matplotlib>=3.3.0->ultralytics) (1.17.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.12/dist-packages (from
sympy>=1.13.3->torch>=1.8.0->ultralytics) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.12/dist-packages (from jinja2->torch>=1.8.0->ultralytics)
(3.0.3)

image 1/1 /PIZA.jpg: 384x640 2 persons, 3 pizzas, 128.5ms
Speed: 2.7ms preprocess, 128.5ms inference, 1.2ms postprocess per image at shape
(1, 3, 384, 640)



```
[70]: !pip install ultralytics
      from ultralytics import YOLO

      # Load a tiny pre-trained model
      yolo_model = YOLO("yolov8n.pt") # n = nano (fast, small)

      # Run detection on your hot dog image
      results = yolo_model("/PPCC.jpg")

      # Show results (it will draw boxes around objects it recognizes)
      results[0].show()
```

```
Requirement already satisfied: ultralytics in /usr/local/lib/python3.12/dist-
packages (8.4.46)
Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.12/dist-
packages (from ultralytics) (2.0.2)
Requirement already satisfied: matplotlib>=3.3.0 in
/usr/local/lib/python3.12/dist-packages (from ultralytics) (3.10.0)
Requirement already satisfied: opencv-python>=4.6.0 in
/usr/local/lib/python3.12/dist-packages (from ultralytics) (4.13.0.92)
Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.12/dist-
packages (from ultralytics) (11.3.0)
Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.12/dist-
packages (from ultralytics) (6.0.3)
Requirement already satisfied: requests>=2.23.0 in
/usr/local/lib/python3.12/dist-packages (from ultralytics) (2.32.4)
Requirement already satisfied: scipy>=1.4.1 in /usr/local/lib/python3.12/dist-
packages (from ultralytics) (1.16.3)
Requirement already satisfied: torch>=1.8.0 in /usr/local/lib/python3.12/dist-
packages (from ultralytics) (2.10.0+cpu)
Requirement already satisfied: torchvision>=0.9.0 in
/usr/local/lib/python3.12/dist-packages (from ultralytics) (0.25.0+cpu)
```

Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (5.9.5)

Requirement already satisfied: polars>=0.20.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (1.35.2)

Requirement already satisfied: ultralytics-thop>=2.0.18 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.0.19)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (1.3.3)

Requirement already satisfied: cycycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (4.62.1)

Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (1.5.0)

Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (26.1)

Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (3.3.2)

Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (2.9.0.post0)

Requirement already satisfied: polars-runtime-32==1.35.2 in /usr/local/lib/python3.12/dist-packages (from polars>=0.20.0->ultralytics) (1.35.2)

Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (3.4.7)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (3.13)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (2.5.0)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (2026.4.22)

Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (3.29.0)

Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (4.15.0)

Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (75.2.0)

Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-

packages (from torch>=1.8.0->ultralytics) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in
/usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages
(from torch>=1.8.0->ultralytics) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-
packages (from torch>=1.8.0->ultralytics) (2025.3.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-
packages (from python-dateutil>=2.7->matplotlib>=3.3.0->ultralytics) (1.17.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.12/dist-packages (from
sympy>=1.13.3->torch>=1.8.0->ultralytics) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.12/dist-packages (from jinja2->torch>=1.8.0->ultralytics)
(3.0.3)

image 1/1 /PPCC.jpg: 480x640 1 truck, 1 pizza, 189.1ms

Speed: 3.3ms preprocess, 189.1ms inference, 1.2ms postprocess per image at shape
(1, 3, 480, 640)

